Software Project Management 4th Edition

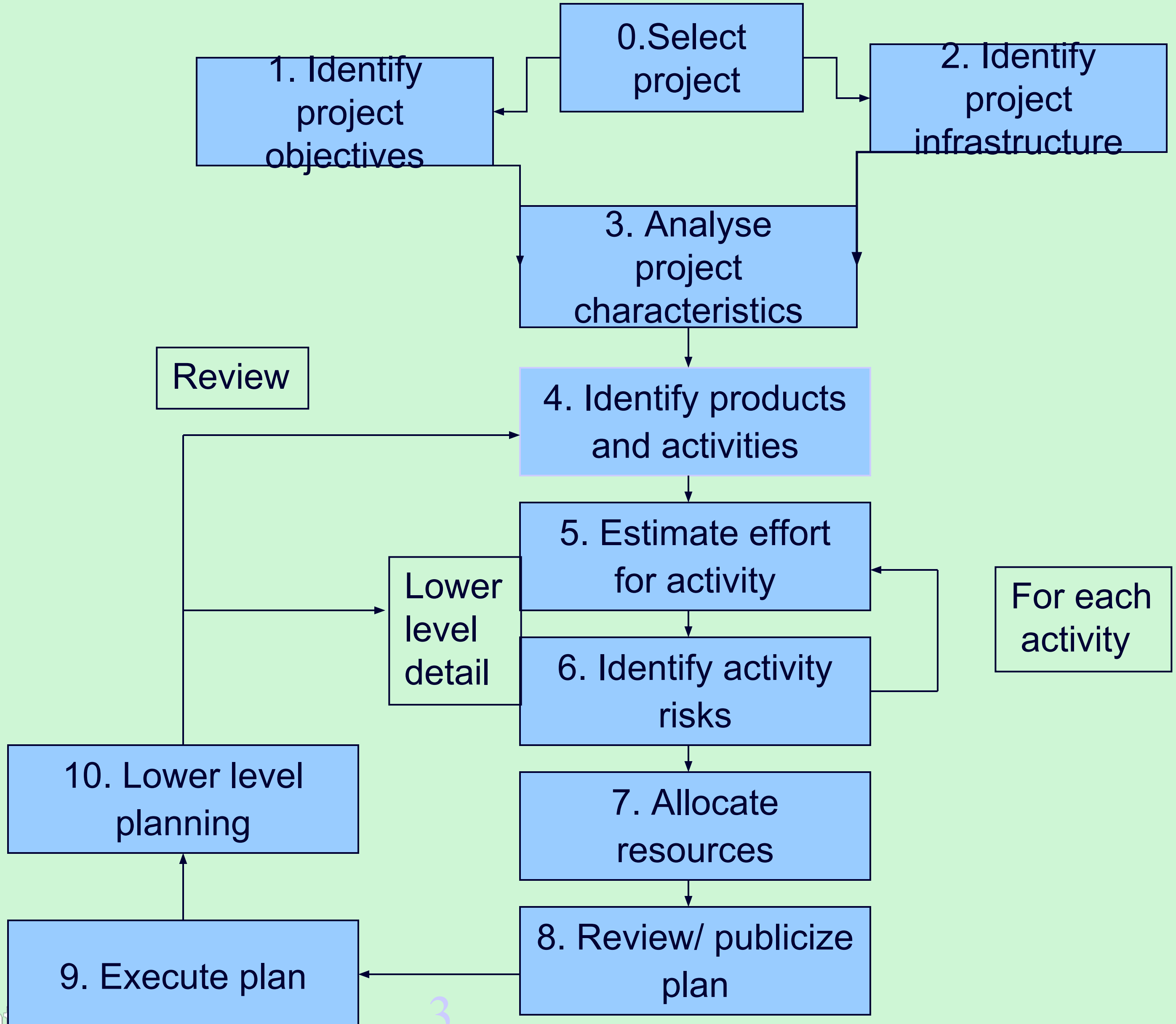
Chapter 2

Step Wise: An approach to planning software projects

‘Step Wise’ - aspirations

- Practicality
  - tries to answer the question ‘what do I do now?’
- Scalability
  - useful for small project as well as large
- Range of application
- Accepted techniques
  - e.g. borrowed from PRINCE etc

'Step Wise' - an overview



A project scenario

- Hardware/software engineering company (C++ language of choice)
- teams are selected for individual projects - some friction has been found between team members
- HR manager suggests psychometric testing to select team

Project scenario - continued

- Software package to be used to test staff
- Visual basic suggested as a vehicle for implementation
- usability is important - decision to carry out usability tests

Step 1 establish project scope and objectives

- 1.1 Identify objectives and measures of effectiveness
  - 'how do we know if we have succeeded?'
- 1.2 Establish a project authority
  - 'who is the boss?'
- 1.3 Identify all stakeholders in the project and their interests
  - 'who will be affected/involved in the project?'

Step 1 continued

- 1.4 Modify objectives in the light of stakeholder analysis
  - ‘do we need to do things to win over stakeholders?’

- 1.5 Establish methods of communication with all parties
  - ‘how do we keep in contact?’

Back to the scenario

- Project authority
  - should be a project manager rather than HR manager?
- Stakeholders
  - project team members to complete on-line questionnaires: concern about results?
- Revision to objectives
  - provide feedback to team members on results

Step 2 Establish project infrastructure

- 2.1 Establish link between project and any strategic plan
  - ‘why did they want the project?’
- 2.2 Identify installation standards and procedures
  - ‘what standards do we have to follow?’
- 2.3. Identify project team organization
  - ‘where do I fit in?’

Step 3 Analysis of project characteristics

- 3.1 Distinguish the project as either objective or product-based.
  – Is there more than one way of achieving success?

- 3.2 Analyse other project characteristics (including quality based ones)
  – what is different about this project?

Step 3 continued

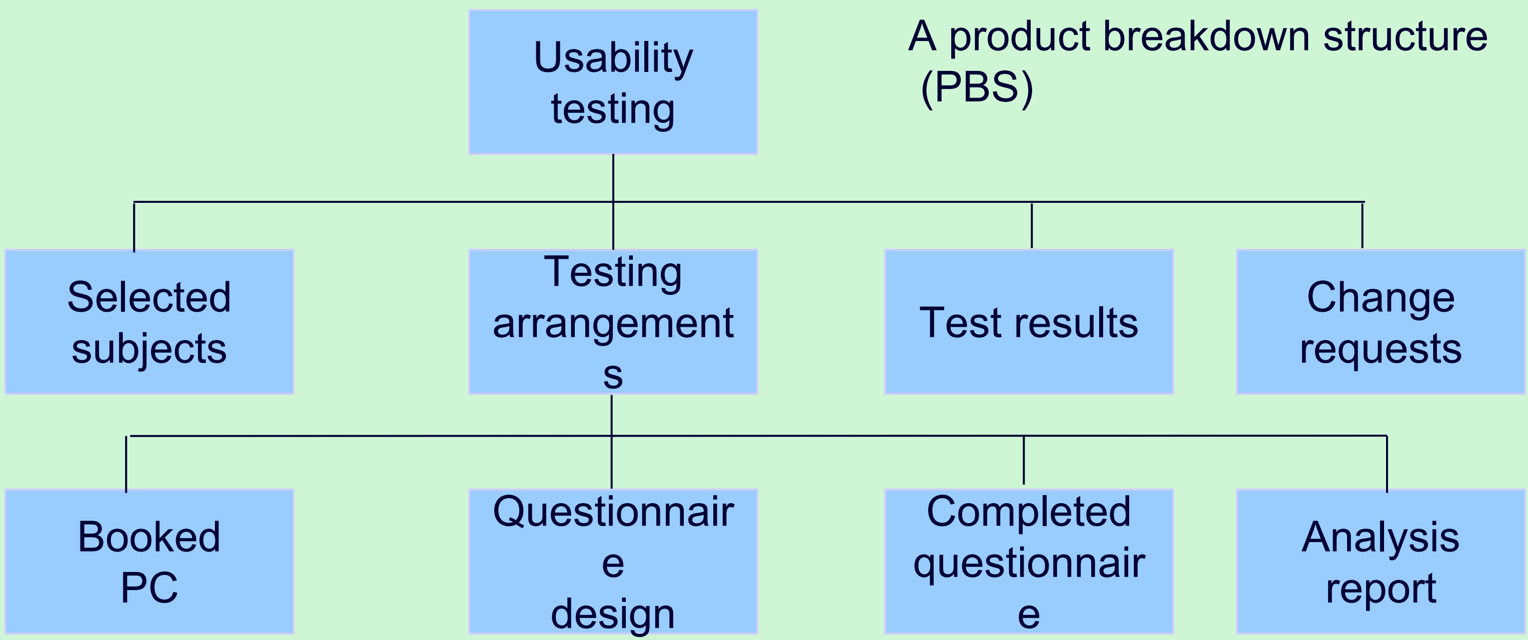
- Identify high level project risks
  - 'what could go wrong?'
  - 'what can we do to stop it?'
- Take into account user requirements concerning implementation
- Select general life cycle approach
  - waterfall? Increments? Prototypes?
- Review overall resource estimates
  - 'does all this increase the cost?'

Back to the scenario

- Objectives vs. products
  – use paper questionnaire then input results of the analysis?
- Some risks
  – team members worried about implications and do no co-operate
  – project managers unwilling to try out application
  – Developer not familiar with features of VB
- Answer? - evolutionary prototype?

Step 4 Identify project products and activities

4.1 Identify and describe project products - ‘what do we have to produce?’



A product breakdown structure (PBS)

Products

- The result of an activity

- Could be (among other things)
  - physical thing (‘installed pc’),
  - a document (‘logical data structure’)
  - a person (‘trained user’)
  - a new version of an old product (‘updated software’)

Products

- The following are NOT normally products:
  - activities (e.g. ‘training’)
  - events (e.g. ‘interviews completed’)
  - resources and actors (e.g. ‘software developer’) - may be exceptions to this
- Products CAN BE *deliverable* or *intermediate*

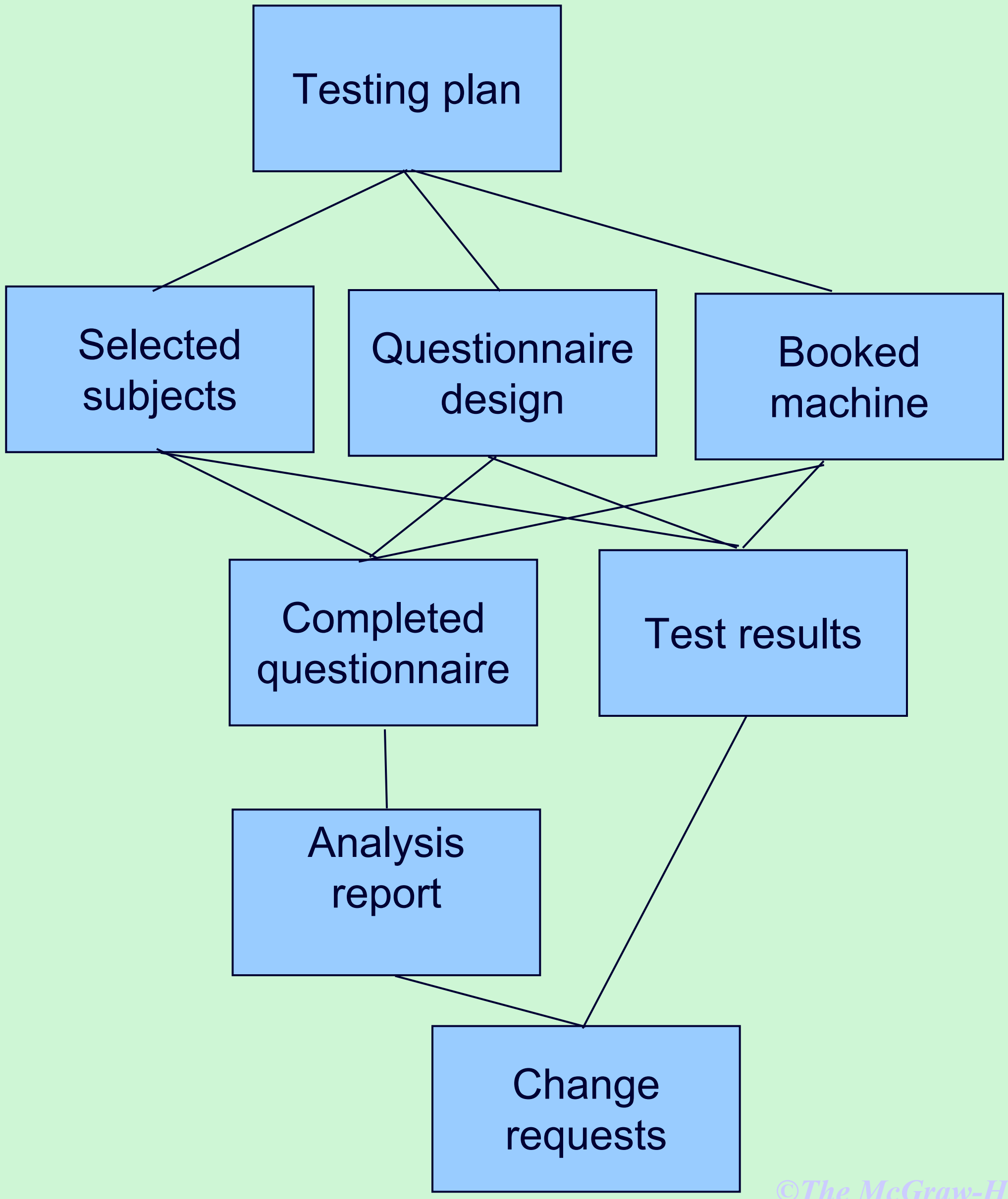
Product description (PD)

- Product identity
- Description - what is it?
- Derivation - what is it based on?
- Composition - what does it contain?
- Format
- Relevant standards
- Quality criteria

Create a PD for 'test data'

Step 4 continued

4.2 document Generic product flows



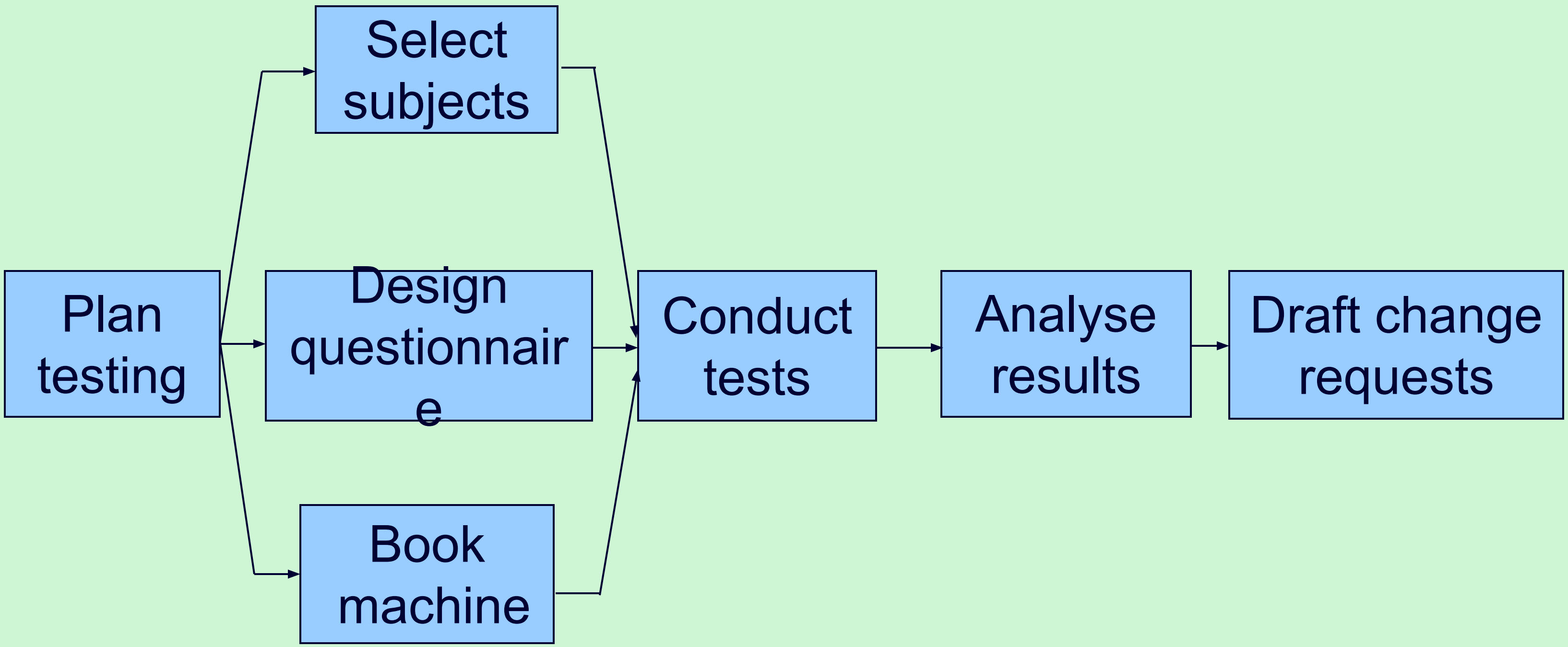
Step 4.3 Recognize product instances

- The PBS and PFD will probably have identified generic products e.g. ‘software modules’
- It might be possible to identify specific instances e.g. ‘module A’, ‘module B’ …
- But in many cases this will have to be left to later, more detailed, planning

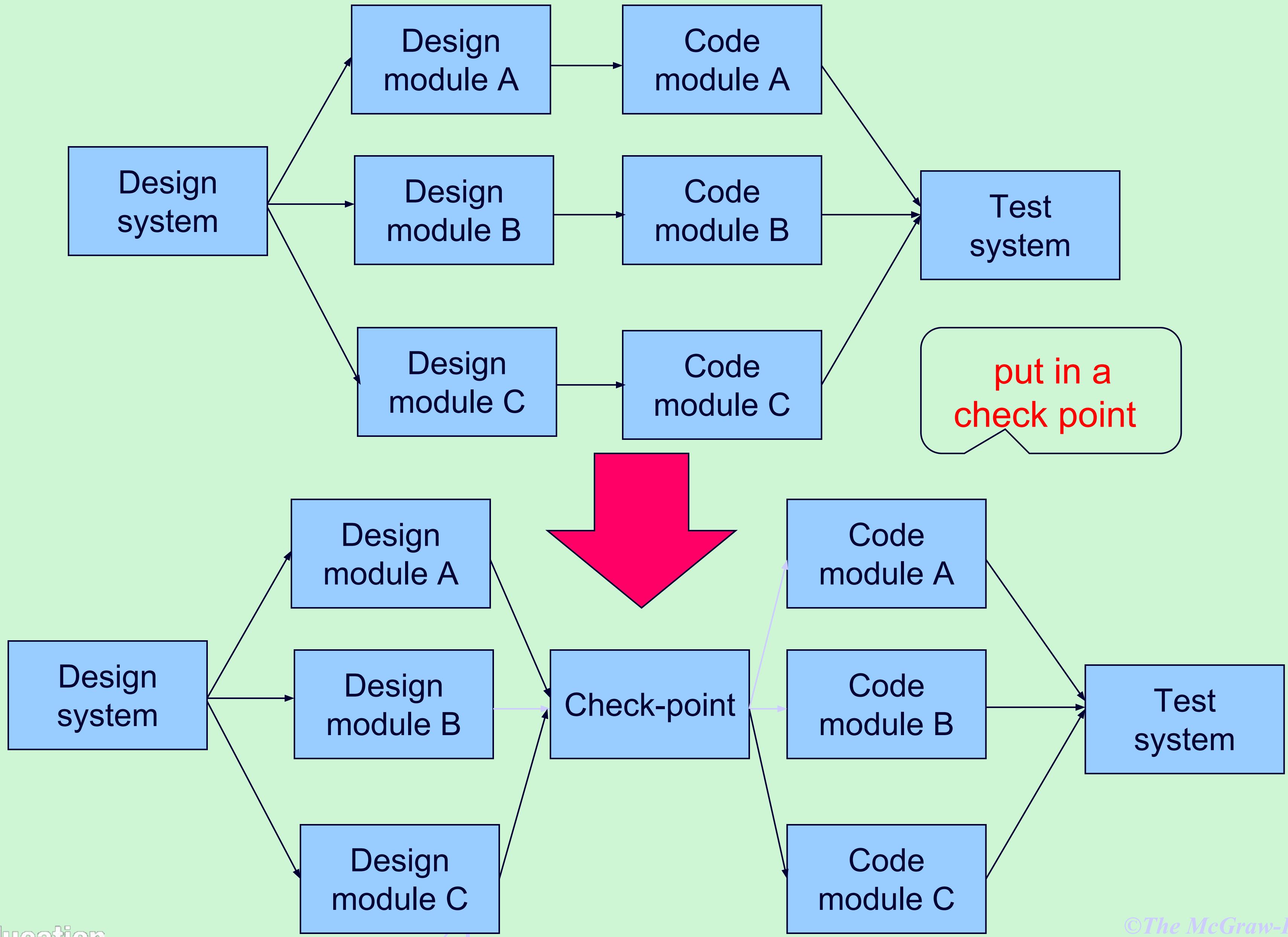
4.4. Produce ideal activity network

- Identify the activities needed to create each product in the PFD
- More than one activity might be needed to create a single product
- Hint: Identify activities by verb + noun but avoid 'produce…' (too vague)
- Draw up activity network

An ‘ideal’ activity



Step 4.5 Add check-points if needed



Step 5:Estimate effort for each activity

- 5.1 Carry out bottom-up estimates
  - distinguish carefully between *effort* and *elapsed* time
- 5.2. Revise plan to create controllable activities
  - break up very long activities into a series of smaller ones
  - bundle up very short activities (create check lists?)

Step 6: Identify activity risks

- 6.1.Identify and quantify risks for activities
  - damage if risk occurs (measure in time lost or money)
  - likelihood if risk occurring
- 6.2. Plan risk reduction and contingency measures
  - risk reduction: activity to stop risk occurring
  - contingency: action if risk does occur

- 6.3 Adjust overall plans and estimates to take account of risks
  - e.g. add new activities which reduce risks associated with other activities e.g. training, pilot trials, information gathering

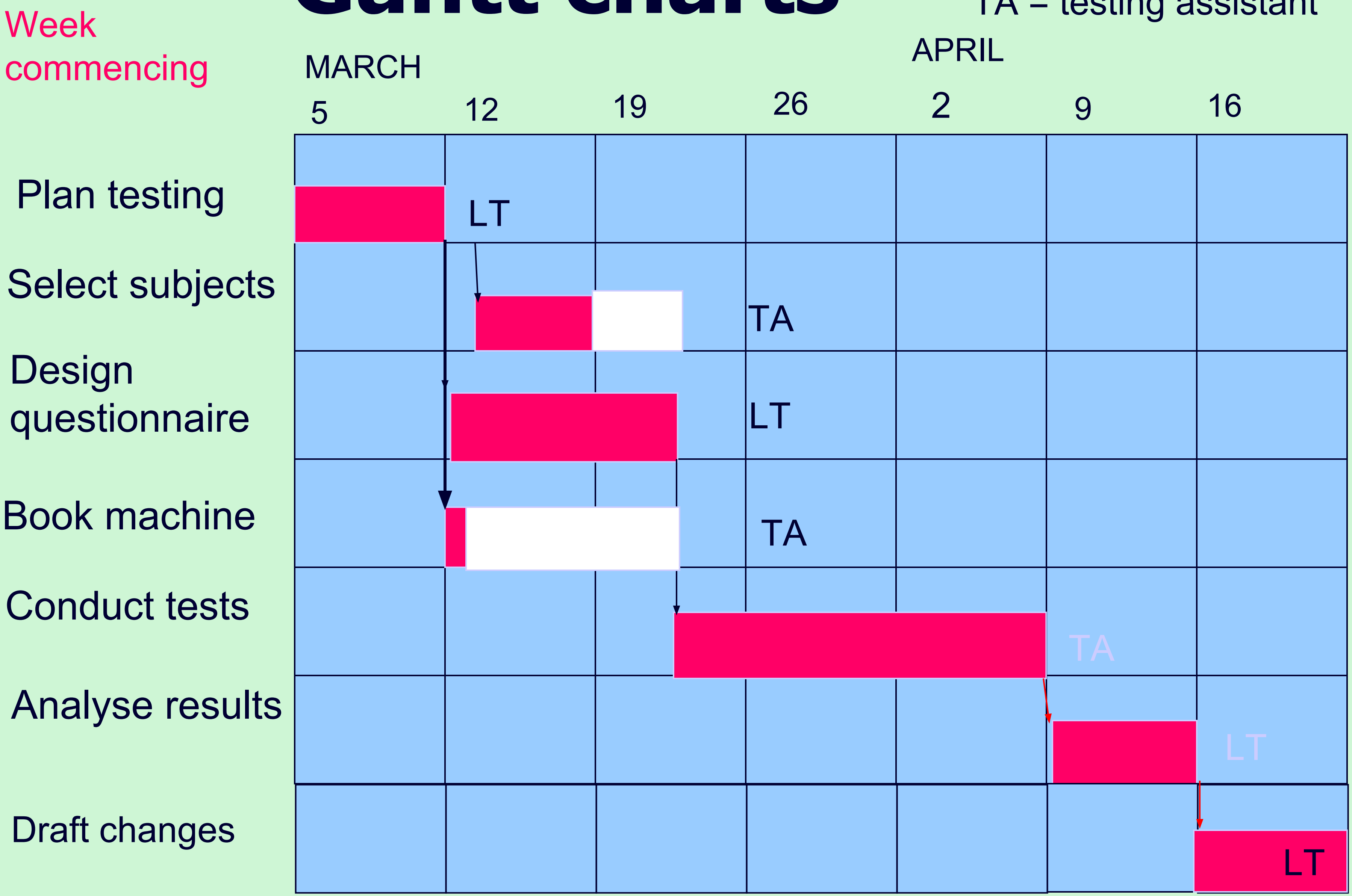
Step 7: Allocate resources

- 7.1 Identify and allocate resources to activities
- 7.2 Revise plans and estimates to take into account resource constraints
  - e.g. staff not being available until a later date
  - non-project activities

Gantt charts

LT = lead tester

TA = testing assistant



Step 8: Review/publicise plan

- 8.1 Review quality aspects of project plan
- 8.2 Document plan and obtain agreement

Step 9 and 10: Execute plan and create lower level plans